

SQL: Comprehensive Theory and Database Concepts Study Guide

Module 1: SQL Basics

1. Fundamentals

- **What is SQL?** SQL stands for **Structured Query Language**. It is the standard domain-specific programming language used to manage, query, and manipulate data stored in Relational Database Management Systems (RDBMS). It was originally developed at IBM by Donald D. Chamberlin and Raymond F. Boyce in the 1970s.
- **What are the features of SQL?**
 - **Declarative Nature:** Users specify *what* data they want to retrieve rather than *how* to access it.
 - **High Portability:** Highly standardized (ANSI/ISO SQL), allowing code to work across different database engines with minor modifications.
 - **Robust Security:** Supports granular permission schemas on tables, columns, views, and schemas.
 - **Transactional Control:** Guarantees data consistency through ACID operations.
 - **Comprehensive Dialects:** Supports creation, modification, deletion, and querying of database schemas.
- **What are the advantages of SQL?**
 - Extremely fast retrieval of millions of records using optimization systems.
 - No coding skills required for basic CRUD operations.
 - Interactive shell environments alongside seamless integrations with programming languages (Python, Java, etc.).
 - Standardized language with a massive community.
- **What are the different types of SQL commands?** SQL statements are divided into five sub-languages:
 1. **DDL (Data Definition Language):** Defines and modifies the database schema structure.
 2. **DML (Data Manipulation Language):** Manipulates data values within the tables.
 3. **DQL (Data Query Language):** Focused on retrieving data.
 4. **DCL (Data Control Language):** Manages permissions and user access rights.
 5. **TCL (Transaction Control Language):** Manages transactions and database states.

- **What is DDL?** Data Definition Language is used to create and modify the structure of database objects (tables, indexes, views, users).
 - *Commands:* CREATE , ALTER , DROP , TRUNCATE , RENAME .
 - [!NOTE] > DDL operations are auto-committed instantly in most database engines (like Oracle, MySQL), meaning they cannot be rolled back.
- **What is DML?** Data Manipulation Language is used to manage the actual data records stored within the database tables.
 - *Commands:* INSERT , UPDATE , DELETE , MERGE .
- **What is DQL?** Data Query Language retrieves data from database objects.
 - *Commands:* SELECT .
- **What is DCL?** Data Control Language regulates privileges and permissions for database users and security roles.
 - *Commands:* GRANT (give access), REVOKE (remove access).
- **What is TCL?** Transaction Control Language manages transactional boundaries, grouping DML queries together to maintain database consistency.
 - *Commands:* COMMIT (save changes), ROLLBACK (undo changes), SAVEPOINT (set an undo checkpoint).
- **Difference between SQL and MySQL?**
 - **SQL** is a query language used to interact with databases.
 - **MySQL** is a relational database management system software (RDBMS) that uses SQL under the hood.
- **Difference between SQL and NoSQL?**
 - **SQL (Relational):** Structured table format, fixed schema, scales vertically (CPU/RAM), guarantees ACID, uses SQL.
 - **NoSQL (Non-Relational):** Document, key-value, graph, or column store, dynamic/schemaless, scales horizontally (sharding), follows CAP theorem (BASE model).

Module 2: Database Fundamentals

- **What is DBMS?** A **Database Management System** is a software interface that manages, stores, retrieves, and updates files/data on disk. It acts as an interface between databases and end-users.
- **What is RDBMS?** A **Relational Database Management System** is an advanced DBMS based on the relational model introduced by E.F. Codd. Data is organized into tables consisting of columns and rows, and relationships are maintained using keys.
- **DBMS vs RDBMS**

- **DBMS** stores data in flat files or tree-like hierarchies, does not enforce relationship integrity, supports single users or basic sharing, and has lower security.
 - **RDBMS** stores data in tables, enforces keys (primary and foreign) to preserve relationships, supports concurrent access, and guarantees ACID compliance.
 - **What is a Database?** An organized collection of structured information or data, typically stored electronically in a computer system.
 - **What is a Table?** A collection of related data entries structured in columns (fields) and rows (records) within a database.
 - **What is a Record?** A single row in a table containing a complete set of attributes for a single entry.
 - **What is a Column?** A vertical partition in a table representing a single attribute or field type.
 - **What is Schema?** The logical structure or blueprint of the database. It defines tables, fields, relationships, views, indexes, types, and constraints.
 - **What is Metadata?** "Data about data." It refers to system schema descriptions, table configurations, indexes, and database statistics stored in system-level system catalogs (e.g., `INFORMATION_SCHEMA`).
-

Module 3: Constraints

- **What is Primary Key?** A column (or combination of columns) that uniquely identifies each row in a table. It cannot contain duplicate values and cannot contain `NULL` values. Only one Primary Key is allowed per table.
- **What is Foreign Key?** A column (or combination of columns) that establishes a link between data in two tables. It references the Primary Key (or Unique Key) of another table, enforcing referential integrity.
- **Primary Key vs Unique Key**
 - A table can have only one Primary Key, but multiple Unique Keys.
 - Primary Key columns cannot accept `NULL` values. Unique Key columns can accept `NULL` values (exactly one `NULL` in SQL Server, multiple `NULL` values in MySQL/PostgreSQL).
- **Candidate Key** A minimal set of attributes that can uniquely identify a record in a table. A table can have multiple Candidate Keys. One is chosen as the Primary Key; the rest are called Alternate Keys.
- **Super Key** A set of one or more attributes that collectively identify unique records in a table. A Candidate Key is a minimal Super Key (a super key with no redundant attributes).
- **Composite Key** A primary or unique key that consists of two or more columns to guarantee uniqueness.
- **Alternate Key** All Candidate Keys that were not selected to be the Primary Key.
- **What is NOT NULL?** A constraint ensuring that a column cannot store a `NULL` value.

- **What is DEFAULT?** A constraint that inserts a specified fallback value when no value is explicitly supplied for the column during an `INSERT` operation.
 - **What is CHECK Constraint?** Enforces domain integrity by limiting values that can be inserted into a column based on a boolean expression (e.g., `CHECK (salary > 0)`).
 - **What is UNIQUE Constraint?** Ensures that all values in a column are unique across the table, preventing duplicates.
-

Module 4: SQL Commands

- **CREATE:** Instantiates new database structures like databases, tables, views, indexes, or procedures: `sql CREATE TABLE Employees ( EmpID INT PRIMARY KEY, Name VARCHAR(100) NOT NULL, Salary DECIMAL(10,2) CHECK (Salary > 0) );`
 - **ALTER:** Modifies the structure of an existing table or object: `sql ALTER TABLE Employees ADD Email VARCHAR(255) UNIQUE;`
 - **DROP:** Deletes a table, view, or database permanently, deleting both schema structure and data: `sql DROP TABLE Employees;`
 - **TRUNCATE:** Deletes all records from a table. The structure remains intact. It is faster than `DELETE` because it releases storage allocations directly without logging individual row deletes: `sql TRUNCATE TABLE Employees;`
 - **RENAME:** Renames an existing table or column.
 - **INSERT:** Adds new rows of data into a table: `sql INSERT INTO Employees (EmpID, Name, Salary) VALUES (1, 'Alice', 75000.00);`
 - **UPDATE:** Modifies existing records based on conditions: `sql UPDATE Employees SET Salary = Salary * 1.10 WHERE EmpID = 1;`
 - **DELETE:** Removes specific rows from a table based on a condition. Logs every row deleted: `sql DELETE FROM Employees WHERE EmpID = 1;`
 - **SELECT:** Retrieves data from one or more tables: `sql SELECT Name, Salary FROM Employees;`
-

Module 5: SQL Clauses

- **WHERE:** Filters records before any grouping is performed.
- **ORDER BY:** Sorts the output records in ascending (`ASC`, default) or descending (`DESC`) order: `sql SELECT * FROM Employees ORDER BY Salary DESC;`
- **GROUP BY:** Groups rows that share the same values in specified columns into summary rows (used with aggregate functions like `SUM`, `COUNT`): `sql SELECT Department, COUNT(*) FROM Employees GROUP BY Department;`
- **HAVING:** Filters group results *after* `GROUP BY` has run. It is used instead of `WHERE` for conditions involving aggregate values: `sql SELECT Department, AVG(Salary) FROM Employees GROUP BY Department HAVING AVG(Salary) > 60000;`

- **DISTINCT**: Removes duplicate rows from the query output.
 - **LIMIT**: Restricts the maximum number of rows returned in the result set.
 - **OFFSET**: Skips a specified number of rows before beginning to return records (essential for pagination queries).
 - **Alias**: Renames a column or table temporarily using `AS` for the duration of a query: `sql SELECT Name AS EmployeeName FROM Employees;`
-

Module 6: Operators

- **Arithmetic Operators**: `+`, `-`, `*`, `/`, `%` (modulo).
 - **Comparison Operators**: `=`, `≠` or `<>` (not equal), `<`, `>`, `≤`, `≥`.
 - **Logical Operators**: `AND` (both conditions true), `OR` (at least one true), `NOT` (negates condition).
 - **BETWEEN**: Checks if a value is within a specified range (inclusive): `sql SELECT * FROM Employees WHERE Salary BETWEEN 50000 AND 80000;`
 - **IN**: Checks if a value matches any value in a list or subquery: `sql SELECT * FROM Employees WHERE Department IN ('HR', 'IT', 'Sales');`
 - **LIKE**: Compares columns against string pattern formats using wildcards:
 - `%`: Matches zero or more characters.
 - `_`: Matches exactly one character. `sql SELECT * FROM Employees WHERE Name LIKE 'A%';` # Starts with A
 - **EXISTS**: Evaluates to `True` if a subquery returns one or more records.
 - **ANY**: Compares a value to any value in the subquery results (equivalent to matching at least one element).
 - **ALL**: Compares a value to all values in the subquery results (requires matching every single element).
-

Module 7: SQL Functions

1. Aggregate Functions

Operate on multiple values across rows to return a single value (ignores `NULL`s, except `COUNT(*)`):

- * `COUNT()`: Returns the number of rows.
- * `SUM()`: Returns the sum of values.
- * `AVG()`: Returns the average value.
- * `MAX()`: Returns the largest value.
- * `MIN()`: Returns the smallest value.

2. String Functions

- `UPPER(str)` / `LOWER(str)`: Converts string casing.
- `LENGTH(str)` or `LEN(str)`: Returns the number of characters.

- `CONCAT(s1, s2, ...)` : Combines strings.
- `SUBSTRING(str, pos, len)` : Extracts `len` characters starting at position `pos`.
- `REPLACE(str, from_str, to_str)` : Replaces occurrences of a substring.
- `TRIM(str)` : Removes leading and trailing spaces.

3. Date Functions

- `CURRENT_DATE` : Returns the current system date.
- `CURRENT_TIMESTAMP` or `NOW()` : Returns the current date and time.
- `DATE_ADD(date, INTERVAL value unit)` : Adds a time interval to a date.
- `DATEDIFF(d1, d2)` : Returns difference in days between two dates.

4. Numeric Functions

- `ROUND(number, decimals)` : Rounds a number to a specified number of decimal places.
- `CEIL(number)` / `CEILING()` : Returns the smallest integer greater than or equal to the number.
- `FLOOR(number)` : Returns the largest integer less than or equal to the number.
- `ABS(number)` : Returns the absolute value of a number.

Module 8: Joins

- **What is JOIN?** An operation used to combine columns from two or more tables based on a related column between them.
 - **INNER JOIN** Returns records that have matching values in both tables. `sql SELECT E.Name, D.DeptName FROM Employees E INNER JOIN Departments D ON E.DeptID = D.DeptID;`
 - **LEFT (OUTER) JOIN** Returns all records from the left table, and matching records from the right table. Non-matching rows on the right are filled with `NULL`.
 - **RIGHT (OUTER) JOIN** Returns all records from the right table, and matching records from the left table. Non-matching rows on the left are filled with `NULL`.
 - **FULL OUTER JOIN** Returns all records when there is a match in either left or right table. Fills unmatched fields with `NULL`.
 - **CROSS JOIN** Returns the Cartesian product of the two tables, combining every row of the left table with every row of the right table (produces $M \times N$ rows).
 - **SELF JOIN** A join in which a table is joined with itself. Useful for referencing hierarchy structures: `sql SELECT E.Name AS Employee, M.Name AS Manager FROM Employees E LEFT JOIN Employees M ON E.ManagerID = M.EmpID;`
 - **NATURAL JOIN** An implicit join that automatically links tables based on all columns sharing matching names and compatible data types.
-

Module 9: Set Operators

Combine the results of two or more SELECT queries.

[!IMPORTANT] Both SELECT queries must have the same number of columns in the same order, with compatible data types.

- **UNION:** Merges result sets, removing duplicate rows.
 - **UNION ALL:** Merges result sets, retaining all duplicate rows (faster because it skips duplicate sorting).
 - **INTERSECT:** Returns only rows that are common to both SELECT queries.
 - **EXCEPT / MINUS:** Returns rows from the first query that are not present in the second query.
-

Module 10: Subqueries

- **What is Subquery?** A query nested inside another query (the outer query). Also known as an inner query.
 - **Types of Subquery**
 - **Single-row Subquery:** Returns at most one row, used with scalar operators (= , < , >).
 - **Multi-row Subquery:** Returns one or more rows, used with set operators (IN , ANY , ALL).
 - **Correlated Subquery** A subquery that references columns from the outer query. The database engine executes the subquery repeatedly, once for each row processed by the outer query: `sql SELECT * FROM Employees E WHERE Salary > (SELECT AVG(Salary) FROM Employees WHERE DeptID = E.DeptID);`
 - **Nested Subquery** An independent subquery. The inner query is executed once first, and its result is supplied to the outer query.
 - **EXISTS / NOT EXISTS** Operators that check for the presence or absence of records in a subquery. They return `True` or `False` and are highly optimized because the database stops scanning the subquery as soon as the first match is found.
-

Module 11: Views

- **What is View?** A virtual table whose contents are defined by a saved SQL query. It does not store physical data on disk (acts as a dynamic window to the underlying tables).
- **Advantages of View**
 - Simplifies complex joins and queries.
 - Improves security by restricting access to sensitive columns (e.g., hiding salary column).

- Provides schema abstraction (data independence).
 - **Materialized View** A view that physically stores query results on disk. It must be updated (refreshed) when underlying tables change. Used in Data Warehouses to speed up slow aggregation queries over massive datasets.
 - **View vs Table** A Table stores physical data. A View is a virtual, query-based structure that retrieves data dynamically at run-time from physical tables.
-

Module 12: Indexes

- **What is Index?** A database structure created on columns to speed up data retrieval operations (searching and sorting) at the cost of slower writes and additional disk space.
 - **Clustered Index** Determines the physical sorting order of rows in the table based on the index key. A table can have only **one** clustered index because physical rows can only be sorted in one order. Primary keys automatically create a clustered index in most engines.
 - **Non-Clustered Index** A separate pointer structure pointing back to the physical data rows. The index key points to the clustered index key or row locator. A table can have **multiple** non-clustered indexes.
 - **Advantages of Index**
 - Drastically reduces query times for `WHERE` , `JOIN` , `ORDER BY` , and `GROUP BY` statements.
 - **Disadvantages of Index**
 - Consumes additional storage space.
 - Slows down DML operations (`INSERT` , `UPDATE` , `DELETE`) because the database must update the index structure (B-Tree) for every write.
-

Module 13: Stored Procedures

- **Stored Procedure** A precompiled collection of SQL statements stored on the database server.
- **Advantages**
 - **Performance:** Precompiled execution plans speed up run times.
 - **Reduced Network Traffic:** Clients send a single procedure call (e.g., `CALL GetReports()`) instead of hundreds of lines of SQL.
 - **Security:** Users can be granted execution permissions on a procedure without having access to the underlying tables.
- **Procedure vs Function**
 - Procedures can run DML statements (`INSERT` / `UPDATE`) and control transactions (`COMMIT` / `ROLLBACK`). Functions cannot.

- Functions *must* return a value. Procedures are not required to return values (they use `OUT` parameters instead).
 - Functions can be called inside SELECT statements (e.g. `SELECT MyFunc(col)`). Procedures must be run with `CALL` or `EXECUTE`.
-

Module 14: Triggers

- **Trigger** A database object that automatically executes in response to DML operations (`INSERT`, `UPDATE`, `DELETE`) on a specific table.
 - **BEFORE Trigger** Runs before the DML statement is executed. Used to validate or modify values before they are written.
 - **AFTER Trigger** Runs after the DML statement completes. Used for auditing, logging changes, or maintaining synchronized tables.
 - **INSTEAD OF Trigger** Intercepts the DML statement and executes alternative logic instead. Commonly used on complex views to make them updatable.
-

Module 15: Transactions

A logical unit of database work consisting of one or more statements. * **COMMIT**: Saves all changes made in the transaction permanently to disk, releasing locks. * **ROLLBACK**: Reverts all changes since the transaction started, restoring the database to its previous state. * **SAVEPOINT**: Establishes a checkpoint in a transaction, allowing you to roll back parts of a transaction instead of the whole thing. * **ACID Properties** Ensures transactional safety: * **Atomicity**: "All or nothing." If any part fails, the whole transaction is aborted. * **Consistency**: A transaction moves the database from one valid state to another, maintaining all schema constraints. * **Isolation**: Transactions execute independently without concurrent interference. * **Durability**: Committed changes survive crashes and power failures.

Module 16: Normalization

The process of organizing data in a database to reduce redundancy and eliminate anomalies (Insert, Update, Delete anomalies). * **1NF (First Normal Form)**: Data values in each column must be atomic (no arrays or repeating groups). Rows must be unique. * **2NF (Second Normal Form)**: Must be in 1NF, and all non-key attributes must be fully functionally dependent on the primary key, eliminating partial dependency (where an attribute depends on only part of a composite key). * **3NF (Third Normal Form)**: Must be in 2NF, and no transitive dependencies must exist (non-key columns cannot depend on other non-key columns). * **BCNF (Boyce-Codd Normal Form)**: A stronger version of 3NF. For every functional dependency $X \rightarrow Y$, X must be a super key. *

Denormalization The process of intentionally adding redundant data to a database to skip complex joins and improve read performance. Common in OLAP data warehouses.

Module 17: Window Functions

Perform calculations across a set of table rows related to the current row, without collapsing the rows (unlike `GROUP BY`). * **OVER()**: Defines the partition and sorting rules for the window calculations. * **ROW_NUMBER()**: Assigns a unique sequential integer to each row inside the partition starting at 1. * **RANK()**: Assigns rank values. If duplicate values exist, they receive the same rank, and subsequent ranks are skipped (e.g., 1, 2, 2, 4). * **DENSE_RANK()**: Assigns rank values. If duplicate values exist, they receive the same rank, but no ranks are skipped (e.g., 1, 2, 2, 3). * **NTILE(N)**: Divides rows in each partition into `N` groups and assigns group numbers. * **LEAD(col, offset)**: Accesses data from a subsequent row within the partition without a self-join. * **LAG(col, offset)**: Accesses data from a preceding row within the partition without a self-join. * **FIRST_VALUE(col)**: Returns the first value in the sorted window. * **LAST_VALUE(col)**: Returns the last value in the sorted window.

Module 18: CTE (Common Table Expression)

- **What is CTE?** A temporary named result set defined within the execution scope of a single query. Created using the `WITH` keyword: `sql WITH DeptAvg AS ( SELECT DeptID, AVG(Salary) AS AvgSal FROM Employees GROUP BY DeptID ) SELECT E.Name, D.AvgSal FROM Employees E JOIN DeptAvg D ON E.DeptID = D.DeptID;`
 - **Recursive CTE** A CTE that references itself. Essential for querying tree or graph data (hierarchies): `sql WITH RECURSIVE OrgChart AS ( SELECT EmpID, Name, ManagerID, 1 AS Level FROM Employees WHERE ManagerID IS NULL UNION ALL SELECT E.EmpID, E.Name, E.ManagerID, 0.Level + 1 FROM Employees E JOIN OrgChart O ON E.ManagerID = O.EmpID ) SELECT * FROM OrgChart;`
 - **Advantages of CTE**
 - Improves code readability and modularity.
 - Allows recursive querying.
 - Simpler substitute for views and temporary tables.
-

Module 19: Performance Optimization

- **Query Optimization:** Fetching only required columns (`SELECT col1` instead of `SELECT *`), joining tables using indexes, avoiding leading wildcards in `LIKE` queries (which trigger table scans: `LIKE '%abc'`), and replacing nested subqueries with joins where possible.

- **Explain Plan:** Diagnostic command (`EXPLAIN SELECT ...`) showing how the database optimizer will execute the query. It reveals join order, index usages, and estimated row costs.
 - **Index Optimization:** Creating indexes on columns frequently used in `WHERE` filters, `JOIN` keys, and `ORDER BY / GROUP BY` sort keys.
 - **Full Table Scan:** Occurs when the database scans every page of a table to find matching rows. It indicates a missing index on filter columns or outdated statistics.
 - **Execution Plan:** The physical graph showing table scans, index scans, join operators, and cost evaluations chosen by the optimizer to run the query.
-

Module 20: Locks

- **Lock** A mechanism used by database engines to manage concurrent access and prevent data conflicts.
 - **Shared Lock (S)** Used for read operations. Multiple transactions can hold shared locks on a resource simultaneously (readers do not block readers).
 - **Exclusive Lock (X)** Used for write operations (`INSERT` , `UPDATE` , `DELETE`). Only one transaction can hold an exclusive lock, blocking all other readers and writers on that resource.
 - **Deadlock** A state where Transaction A holds Lock 1 and waits for Lock 2, while Transaction B holds Lock 2 and waits for Lock 1. The database engine resolves this by killing one of the transactions (the deadlock victim) and rolling it back.
-

Module 21: SQL Architecture

- **SQL Execution Order** The logical order in which a database evaluates clauses (which differs from written syntax order): ``1. FROM / JOIN → 2. WHERE → 3. GROUP BY → 4. HAVING → 5. SELECT → 6. DISTINCT → 7. ORDER BY → 8. LIMIT / OFFSET ``
 - **Query Processing Steps**
 - **Parser:** Performs syntax checks, validates table and column names (semantic check), and translates the query into an internal representation (query tree).
 - **Optimizer:** Evaluates different query plans (using join order algorithms, index evaluations) and selects the plan with the lowest cost.
 - **Executor:** Receives the query plan and runs the instructions, fetching data from disk pages or buffer cache.
-

Module 22: ACID

- **Atomicity:** Prevents partial updates. If a transaction crashes midway, all modifications made up to that point are rolled back.
 - **Consistency:** Enforces constraints. Schema rules, data types, and primary/foreign keys are checked. If a rule is violated, the transaction is rejected.
 - **Isolation:** Dictates concurrency control. Managed via **Isolation Levels**:
 1. **Read Uncommitted:** Allows dirty reads (reading uncommitted changes of other transactions).
 2. **Read Committed:** Prevents dirty reads.
 3. **Repeatable Read:** Prevents dirty reads and non-repeatable reads (re-reading data returns different values).
 4. **Serializable:** Highest isolation. Prevents all anomalies including phantom reads (re-reading returns new rows) by locking ranges.
 - **Durability:** Ensures committed updates are written to the transaction log (WAL - Write-Ahead Logging) so they can be restored after a power failure.
-

Module 23: SQL Comparisons

Comparison	Option A	Option B	Option C
DELETE vs TRUNCATE vs DROP	DELETE: DML command. Deletes specific rows. Slow. Logs all deletes. Can be rolled back.	TRUNCATE: DDL command. Deletes all rows. Fast. Releases storage pages. Cannot be rolled back in some systems.	DROP: DDL command. Deletes table schema and data permanently.
WHERE vs HAVING	WHERE: Filters rows before grouping (<code>GROUP BY</code>). Cannot evaluate aggregate functions.	HAVING: Filters group summaries after grouping. Used with aggregate functions.	
CHAR vs VARCHAR	CHAR: Fixed-length string. Pads spaces for unused characters. Faster read access.	VARCHAR: Variable-length string. Uses only needed space plus length bytes. Saves space.	
UNION vs UNION ALL	UNION: Combines queries, sorting and removing duplicate rows. Slower.	UNION ALL: Combines queries, retaining all duplicate rows. Faster.	
Primary Key vs Foreign Key	Primary Key: Uniquely identifies rows. Cannot be NULL. Max 1 per table.	Foreign Key: Link to another table's Primary Key. Prevents orphan records. Can be NULL. Multiple allowed.	
Clustered vs Non-Clustered Index	Clustered Index: Sorts physical data rows on disk. Max 1 per table.	Non-Clustered Index: Separate index structure pointing back to data. Multiple allowed.	
View vs Materialized View	View: Virtual table. Evaluates query dynamically at runtime. Stores no data.	Materialized View: Physical cache storing query results. Speeds up reads. Must be refreshed.	

Comparison	Option A	Option B	Option C
Procedure vs Function	Stored Procedure: Can control transactions (<code>COMMIT</code>) and run DML. Calls with <code>CALL</code> .	Function: Must return a value. No transactions or DML allowed. Calls inside <code>SELECT</code> .	
DBMS vs RDBMS	DBMS: Stores flat files, no relationship enforcement, single-user.	RDBMS: Stores tables, enforces primary/foreign keys, supports concurrent access.	
INNER JOIN vs OUTER JOIN	INNER JOIN: Returns rows only when join key matches in both tables.	OUTER JOIN: Returns matched rows, plus unmatched rows from left/right/both filled with <code>NULL</code> .	
Correlated vs Non-Correlated	Correlated Subquery: References outer query columns. Runs once per outer row. Slower.	Non-Correlated Subquery: Independent query. Runs once first. Faster.	
GROUP BY vs ORDER BY	GROUP BY: Groups identical rows to aggregate them.	ORDER BY: Sorts final result rows.	
RANK vs DENSE_RANK vs ROW_NUMBER	ROW_NUMBER: Unique sequential ID (1, 2, 3, 4).	RANK: Skips ranks on duplicates (1, 2, 2, 4).	DENSE_RANK: No skipped ranks on duplicates (1, 2, 2, 3).
EXISTS vs IN	EXISTS: Stops scanning as soon as first match is found. Best for subqueries.	IN: Scans the entire list. Best for static lookup lists.	
DELETE vs CASCADE DELETE	DELETE: Basic delete. Fails if child tables have references.	CASCADE DELETE: Deletes parent row and automatically deletes all referencing child rows.	

Comparison	Option A	Option B	Option C
OLTP vs OLAP	OLTP (Transactional): Optimized for fast writes, normalized, handles daily operations.	OLAP (Analytical): Optimized for fast reads, denormalized, handles historical queries.	
SQL vs NoSQL	SQL: Relational tables, fixed schema, ACID, scales vertically.	NoSQL: Document/Key-Value, dynamic schema, BASE, scales horizontally.	

Module 24: Database Design

- **ER Diagram Entity-Relationship Diagram.** A visual schematic showing tables (entities), column attributes, and cardinality lines (relationships) of a database.
 - **Cardinality** Defines the quantitative relationship between two tables:
 - **One-to-One (1:1):** A record in Table A relates to exactly one record in Table B (e.g., User and Passport). Model with a unique Foreign Key in either table.
 - **One-to-Many (1:N):** A record in Table A relates to multiple records in Table B (e.g., Department and Employees). Model by placing the Foreign Key in Table B (the "Many" side).
 - **Many-to-Many (M:N):** Records in Table A relate to multiple records in Table B, and vice-versa (e.g., Students and Courses). Model using a **junction table** (bridge table) containing Foreign Keys pointing to both tables.
-

Module 25: Data Warehouse (Frequently Asked)

- **ETL (Extract, Transform, Load)** Data is extracted from source systems, transformed (cleaned, normalized) in a staging server, and loaded into the data warehouse.
- **ELT (Extract, Load, Transform)** Data is extracted and loaded directly into the target database/cloud warehouse (e.g., Snowflake, BigQuery), with transformations performed in-place using target compute resources.
- **Data Warehouse** A centralized database optimized for analytical (OLAP) querying, integrating data from multiple business operational databases.
- **Data Lake** A repository storing raw, unstructured (videos, logs), semi-structured (JSON), and structured data at scale.

- **Fact Table** The central table in a warehouse schema. It stores quantitative metrics (e.g., units sold, total revenue) and foreign keys pointing to dimension tables.
 - **Dimension Table** Surrounds the fact table. Stores descriptive context (e.g., product categories, customer details, dates).
 - **Star Schema** A layout where a central Fact Table connects directly to denormalized Dimension Tables (resembles a star). Highly optimized for fast read times.
 - **Snowflake Schema** A layout where Dimension Tables are normalized into sub-dimension tables. Saves disk space but requires more joins.
-

Module 26: Interview Scenario Questions

1. How do you optimize a slow query?

1. Check the **Execution Plan** (`EXPLAIN`) to locate Bottlenecks (e.g., Full Table Scans).
2. Add missing **Indexes** on columns used in `WHERE` , `JOIN` keys, and `ORDER BY` .
3. Rewrite query: Replace `SELECT *` with specific columns, use `EXISTS` instead of `IN` for subqueries, and replace nested subqueries with Joins where possible.
4. Partition massive tables, build materialized views for aggregations, and optimize hardware/buffer configurations.

2. Why is your query taking too much time?

Typically caused by missing indexes, table scans, lock contentions/deadlocks, outdated statistics, cartesian products (missing join keys), or network bottlenecks from returning excessive rows.

3. When would you use an Index?

Use indexes on columns that are frequently filtered (`WHERE`), sorted (`ORDER BY`), grouped (`GROUP BY`), or used to join tables. Do *not* index columns with very low cardinality (e.g., Gender) or tables that receive high-frequency writes.

4. How do you remove duplicate records?

Use a CTE containing the `ROW_NUMBER()` window function partitioned by the duplicate key columns:

```
WITH CTE_Duplicates AS (  
    SELECT Name, Email,  
           ROW_NUMBER() OVER (PARTITION BY Name, Email ORDER BY ID) AS RowNum  
    FROM Users  
)  
DELETE FROM Users  
WHERE ID IN (SELECT ID FROM CTE_Duplicates WHERE RowNum > 1);
```

5. How do you find the second-highest salary?

Using subquery:

```
SELECT MAX(Salary)
FROM Employees
WHERE Salary < (SELECT MAX(Salary) FROM Employees);
```

Using `LIMIT` and `OFFSET` :

```
SELECT Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 1 OFFSET 1;
```

6. How do you find the nth-highest salary?

Using `DENSE_RANK()` inside a subquery:

```
SELECT Salary FROM (
    SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
    FROM Employees
) WHERE rnk = :n;
```

7. How do you retrieve the top 3 salaries department-wise?

```
SELECT DeptID, Name, Salary FROM (
    SELECT DeptID, Name, Salary,
        DENSE_RANK() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS rnk
    FROM Employees
) WHERE rnk ≤ 3;
```

8. How do you detect duplicate records?

Use `GROUP BY` and `HAVING` :

```
SELECT Email, COUNT(*)
FROM Users
GROUP BY Email
HAVING COUNT(*) > 1;
```

9. How do you handle transactions?

Wrap multiple queries inside transactional block statements:

```
BEGIN TRANSACTION;  
UPDATE Accounts SET Balance = Balance - 500 WHERE AccountID = 101;  
UPDATE Accounts SET Balance = Balance + 500 WHERE AccountID = 102;  
-- Verify success  
COMMIT;  
-- If an error occurs, run: ROLLBACK;
```

10. What happens when COMMIT is executed?

All database changes made in the transaction are saved permanently to disk, data modifications become visible to other connections, and locks held on rows/tables are released.

11. What happens when ROLLBACK is executed?

All modifications made since the start of the transaction (or to a savepoint) are undone in memory/log, restoring the previous data state and releasing transactional locks.

12. How would you design a database for an e-commerce application?

Create structured entities: * **Users** (UserID [PK], Email, Password, Name) * **Products** (ProductID [PK], Name, Price, StockQuantity) * **Orders** (OrderID [PK], UserID [FK], OrderDate, TotalAmount) * **OrderItems** (OrderItemID [PK], OrderID [FK], ProductID [FK], Quantity, UnitPrice) * **Payments** (PaymentID [PK], OrderID [FK], PaymentDate, Status) Relationships: * Users to Orders (1:N) * Orders to OrderItems (1:N) * Products to OrderItems (1:N) * Orders to Payments (1:1) Constraints: * Product price and order quantity must be > 0 (CHECK constraint). * User email must be UNIQUE.



Module Summary & Key Takeaways

Theory Focus: Relational Database Design, SQL Query Optimization, Transaction Isolation, and CTEs.

This module reviews foundational database design principles and relational schema optimization. Core learnings include:

- **Schema Normalization:** Structural database anomalies and redundancies are eliminated systematically using normal forms (1NF, 2NF, 3NF, BCNF) to ensure data is decoupled into logical tables linked by keys.
- **Transaction Control (ACID):** All database transactions must conform to Atomicity, Consistency, Isolation, and Durability. Isolation levels (Read Uncommitted, Read Committed, Repeatable Read, Serializable) balance database concurrency against anomalies (dirty, non-repeatable, and phantom reads).
- **Performance & Optimization:** Optimized queries leverage indexes (B-Tree indexes for range queries, Hash indexes for point queries) and execute operations using modern constructs like Common Table Expressions (CTEs) and Window functions (e.g., `ROW_NUMBER()`, `RANK()`, `LEAD()` / `LAG()`) to evaluate values over partitioned rows without collapsing the underlying dataset.